

Testing and Exceptions

Slides for 3/31/25

(Including green slides based on
lecture by Cynthia Lee)

Automated testing

- Nice feature of codingbat: it tells if your code is correct without a human looking at it
- Want same thing for large coding projects so you can tell if changes break part of the code
- Relies on thorough test cases
 - each is a call to your code and expected answer

Unit testing

- Whether you are doing black-box or clear-box testing, you should test every important unit of a software system
- What is a *unit*? In object-oriented programming, usually a software unit is taken to be a single method
- So: we should test every method of every class in the software
- **JUnit** is a widely used framework for unit testing of Java software...

JUnit basics

- To do unit testing in the JUnit framework:
- Mark methods as test cases by putting @Test before them. Each “@Test” method should test a particular thing
- Optional:
 - Define instance variables that store the state of the “test fixture”, i.e. the objects that will be tested
 - Initialize the fixture state with a method marked with @Before
 - Clean-up after a test with a method marked with @After

TestCase and test fixtures

- Import as appropriate
- Define instance variables that store the state of the “test fixture”, i.e. the objects that will be tested

```
import static org.junit.Assert.*;

import org.junit.After;
import org.junit.Before;
import org.junit.Test;

public class RectangleTester {

    private Rectangle r1, r2; // test fixtures
```

setUp() and tearDown()

```
/* Called AUTOMATICALLY before each @Test method is run */
@Before
void setUp() {
    r1 = new Rectangle();
    r2 = new Rectangle(2.0, 3.0);
}

/* Called AUTOMATICALLY after each @Test method is run */
@After
void tearDown() {
    r1 = null;
    r2 = null;
}
```

Make sure each test method starts with a clean copy of the test fixture.

@Before
@Test
@After

This is the sequence of calls the JUnit framework does for you *automatically* for *each* test method @Test that is invoked.

Coding a test case

```
/** Test case 2.1: verify that default constructor sets
    default instance variable values correctly
 */
@Test
public void testDefaultInstance() {
    assertEquals(1.0, r1.getLength());
    assertEquals(1.0, r1.getHeight());
}
```

Remember:

@Before will be called *prior* to each test method getting called, creating test object `r1` anew for each test

```
/** Test case 2.6: verify that mutator for length throws
    * exception on illegal input.
 */
@Test (expected = IllegalArgumentException.class)
public void testMutatorIllegalInput() {
    r1.setLength(0); // 0 is illegal, should throw exception
}
```

More JUnit basics

- Test fixture instance variables are optional: you can use test method local variables instead
- When you run your TestCase class as a program, each @Test method is called automatically...
- If a JUnit assertion fails when a @Test method runs, that test **fails**
- If @Test method throws an exception, that is considered an **error**, not a test failure!
- If a @Test method returns normally, that test **passes**

JUnit assertion methods

- The JUnit framework provides many useful assertion methods to use in your @Test methods

```
assertEquals(x,y) // fail if x is not equal to y  
assertTrue(b) // fail if b has boolean value false  
assertFalse(b) // fail if b has boolean value true  
assertSame(x,y) // fail if x & y point to diff objs  
assertNotSame(x,y) // fail if point to same obj  
assertNotNull(x) // fail if x is not null  
assertNotNull(x) // fail if x is null  
fail() // always fails
```

- All assertion methods are overloaded with a version that takes an additional first argument of type String, for a message

Short version for our class

- Create a new class, normally called *TestSomething*
- Give it the import statements
- Select the test tube and enable tests (select JUnit)
- Declare object references as attributes, create the objects (and set them up) in `@Before`
- Write testing methods (`@Test`) with assertions
 - I almost exclusively use `assertEquals`

Programming by contract

- Class descriptions can be viewed as a contract between the implementer and the user; if the user meets the preconditions, then the method works as advertised
 - Can test this independent of the rest of the code
 - Why not make unit tests for each class **before** it is implemented?

Process for Test-Driven Development

- Write stub code: minimal code with correct signatures that compiles
- Repeatedly:
 - Write test cases for some of desired functionality
 - Implement that functionality
 - Use test cases to make sure it works

Exceptions

- Errors that aren't explained via the method's return value
- Ex: What should `remove()` return when the list is empty?
 - One idea is `null`; implies List does not contain `null`

Recall: Coding a test case

```
/** Test case 2.1: verify that default constructor sets
    default instance variable values correctly
 */
@Test
public void testDefaultInstance() {
    assertEquals(1.0, r1.getLength());
    assertEquals(1.0, r1.getHeight());
}
```

Remember:

@Before will be called *prior* to each test method getting called, creating test object `r1` anew for each test

```
/** Test case 2.6: verify that mutator for length throws
    * exception on illegal input.
 */
@Test (expected = IllegalArgumentException.class)
public void testMutatorIllegalInput() {
    r1.setLength(0); // 0 is illegal, should throw exception
}
```

Calling code with exceptions

```
try {  
    //do stuff that may generate exceptions  
} catch(ExceptionType ex) {  
    //handle the exception  
} catch(AnotherExceptionType ex) {  
    //handle another kind of exception  
}
```

Another way to test that an exception is thrown

```
try {  
    //do stuff that should generate exceptions  
    fail();  
} catch(ExceptionType ex) {  
    //NOTHING HERE  
}
```

Throwing an exception

```
throw new IndexOutOfBoundsException();
```



exceptions are objects

create one of the correct type